

DAT Radar Embed: Monetizing DAT.co Treasury Intelligence

R14725076 Kuan Cheng, Ku

June 2026

Contents

1	GitHub Repository and Demo	4
2	1) Target Customer	4
3	2) Evidence of Demand and Willingness to Pay	4
3.1	2.1 Evidence collection process	4
3.2	2.2 Key evidence	5
3.3	2.3 Willingness-to-pay estimate	5
4	3) Technical System Design	5
4.1	3.1 Data sources	5
4.2	3.2 Ingestion and storage	5
4.3	3.3 Processing	5
4.4	3.4 Delivery	6
4.5	3.5 Architecture diagram	6
5	Bonus) Go-to-Market Difficulties	6
6	Reproducibility and Deliverables Checklist	7

1 GitHub Repository and Demo

- GitHub: <https://github.com/<your-account>/dat-radar-embed>
- Live app: <https://<your-vercel-domain>/>
- Embed demo: <https://<your-vercel-domain>/demo>

2 1) Target Customer

DAT Radar targets **small-to-mid crypto product teams** (3 to 20 people), especially:

- FinTech backend engineers integrating market widgets
- Product managers of crypto media sites who publish treasury analytics
- Independent research tool builders who need reliable DAT.co metrics

Their current workaround is usually one of:

1. Manual spreadsheets updated from SEC filings
2. In-house scripts that break whenever filing format changes
3. Screenshots of third-party charts with weak reproducibility

DAT Radar is a better wedge because it removes the highest-friction engineering task: robust SEC holdings extraction and mNAV calculation.

3 2) Evidence of Demand and Willingness to Pay

3.1 2.1 Evidence collection process

To avoid speculative claims, we created reproducible scripts and raw artifacts:

- `scripts/collect_demand_evidence.py`
- `docs/demand-evidence/raw/competitor_pricing.csv`
- `docs/demand-evidence/raw/hn_signal.csv`
- `docs/demand-evidence/raw/reddit_signal.csv`

Methodology is documented in:

- `docs/demand-evidence/methodology.md`
- `docs/demand-evidence/competitor-pricing.md`

3.2 2.2 Key evidence

1. **Competitor pricing anchors exist** for both API and insight products (e.g., Finnhub, Messari, TradingView widgets).
2. **Community signal exists** around Bitcoin treasury and MSTR valuation topics.
3. **Operational pain is concrete**: maintaining SEC parsers is brittle and costly.

3.3 2.3 Willingness-to-pay estimate

If maintaining an in-house parser takes roughly 40 to 80 engineer-hours over a quarter, a subscription around \$99/month can be economically justified for teams that ship treasury-focused content or dashboards.

Proposed tiers:

- Free: one ticker, watermark, 1k req/day
- Pro: \$99/mo, up to five tickers, higher limits, optional watermark removal
- Enterprise: custom SLA, bulk historical exports

4 3) Technical System Design

4.1 3.1 Data sources

- Yahoo Finance for MSTR and BTC-USD daily prices
- SEC EDGAR 8-K filings for BTC holdings disclosures
- Local static holdings baseline for robust fallback

4.2 3.2 Ingestion and storage

- Python ETL pipeline (pipeline/) runs:
 - `ingest_prices.py`
 - `ingest_holdings.py`
 - `compute_mnav.py`
 - `analytics_mapreduce.py`
- Relational storage in SQLite/PostgreSQL schema (`storage/schema.sql`)
- Parquet archives for reproducible data artifacts

4.3 3.3 Processing

For each trading day:

1. Join MSTR and BTC prices
2. Forward-fill holdings using the latest filing at or before date
3. Compute market cap, treasury value, BTC/share, mNAV
4. Compute rolling mNAV statistics and BTC realized volatility

A map-reduce style analytics script computes cross-series correlation metrics in parallel using Python multiprocessing.

4.4 3.4 Delivery

- Dashboard endpoint: `/api/market-data` (DB-first, live fallback)
- B2B API endpoint: `/api/v1/mnav/[ticker]` with API key
- Embed endpoint: `/embed/[ticker]` (iframe-ready)
- Integration demo: `/demo`

4.5 3.5 Architecture diagram

The end-to-end architecture follows four stages:

1. **Ingestion:** Yahoo Finance (MSTR/BTC prices) and SEC EDGAR (8-K holdings)
2. **Batch processing:** Python ETL (`ingest_*`, `compute_mnav`) and MapReduce-style analytics
3. **Storage:** SQLite/PostgreSQL for serving + Parquet archives for reproducibility
4. **Delivery:** REST API (`/api/v1/mnav`), embed iframe (`/embed/MSTR`), and partner demo page (`/demo`)

A full mermaid diagram is maintained in `docs/architecture.md` in the repository.

5 Bonus) Go-to-Market Difficulties

1. **Trust and adoption:** teams must trust accuracy of parsed holdings.
2. **Data-source policy risk:** external API changes can break ingestion.
3. **Cold start:** only one ticker (MSTR) is useful but narrow.
4. **Competition:** incumbents can bundle similar metrics with broader suites.
5. **Unit economics:** low-priced plans must still cover infra and maintenance.

Mitigations include transparent methodology docs, data lineage logs, and migration path to managed Postgres + scheduled ETL.

6 Reproducibility and Deliverables Checklist

- Source code repo with ingestion, processing, and delivery code
- README with local run + ETL instructions
- Demand-evidence scripts and raw outputs
- Architecture document and report
- Optional deployment URL for grading convenience